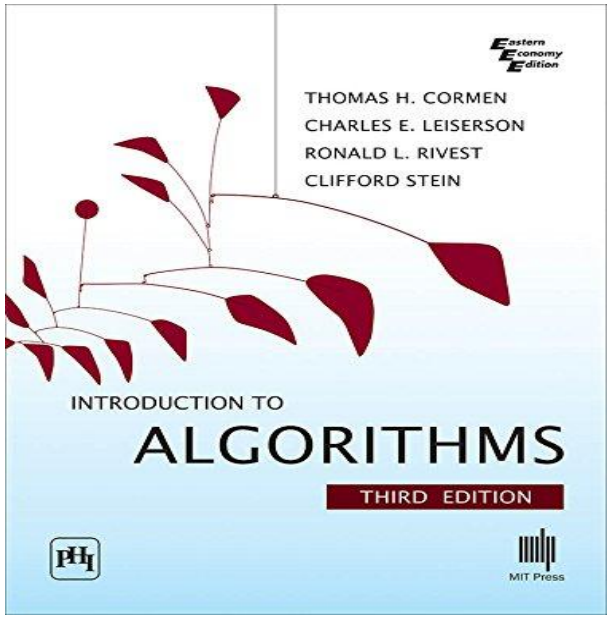
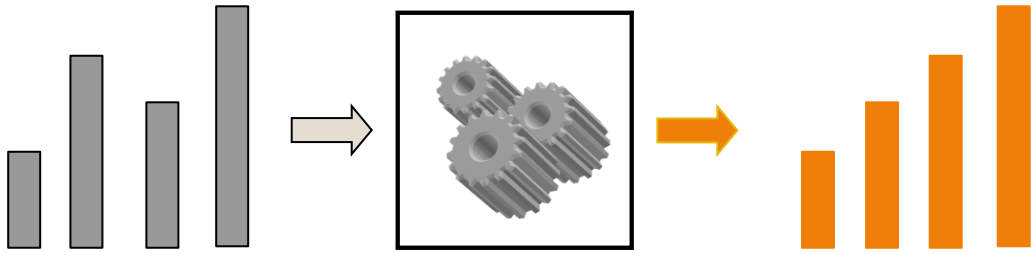
Kuliah 1
Analisis Algoritme

Disampaikan oleh : Dr. Eng. Annisa, S.Kom., M.Kom.

evenhttps://www.youtube.com/watch?v=a4BlnDjeK4Y



Algorithm!



Input

Algorithm 1
Algorithm 2
Algorithm 3
⋮
⋮
Algorithm n

Output

What is Algorithm ?

An **algorithm** is a step-by-step procedure for solving a problem in a finite amount of time.

How to choose best algorithm?

Correctness

Efficiency

Why efficiency is important?



Computers may be fast, but they are not infinitely fast



Memory may be cheap, but it is not free

Bounded Resources!

Computing time and Memory Space are Bounded Resources :
Use them wisely

High Specs always faster?

B runs 20 times faster than A!

$$\frac{2 \cdot (10^6)^2 \text{ instructions}}{10^9 \text{ instructions/second}} = 2000 \text{ seconds}$$



Computer Name	A	B
One million numbers to sort	2000 seconds	100 seconds
Ten million numbers to sort	**2.3 days**	**Under 20 minutes**

$$\frac{50 \cdot 10^6 \lg 10^6 \text{ instructions}}{10^7 \text{ instructions/second}} \approx 100 \text{ seconds}$$



Computer Name	A
Execution rate	One billion instructions per second
Algorithm	Insertion sort ($2n^2$)
Numbers to sort	one million

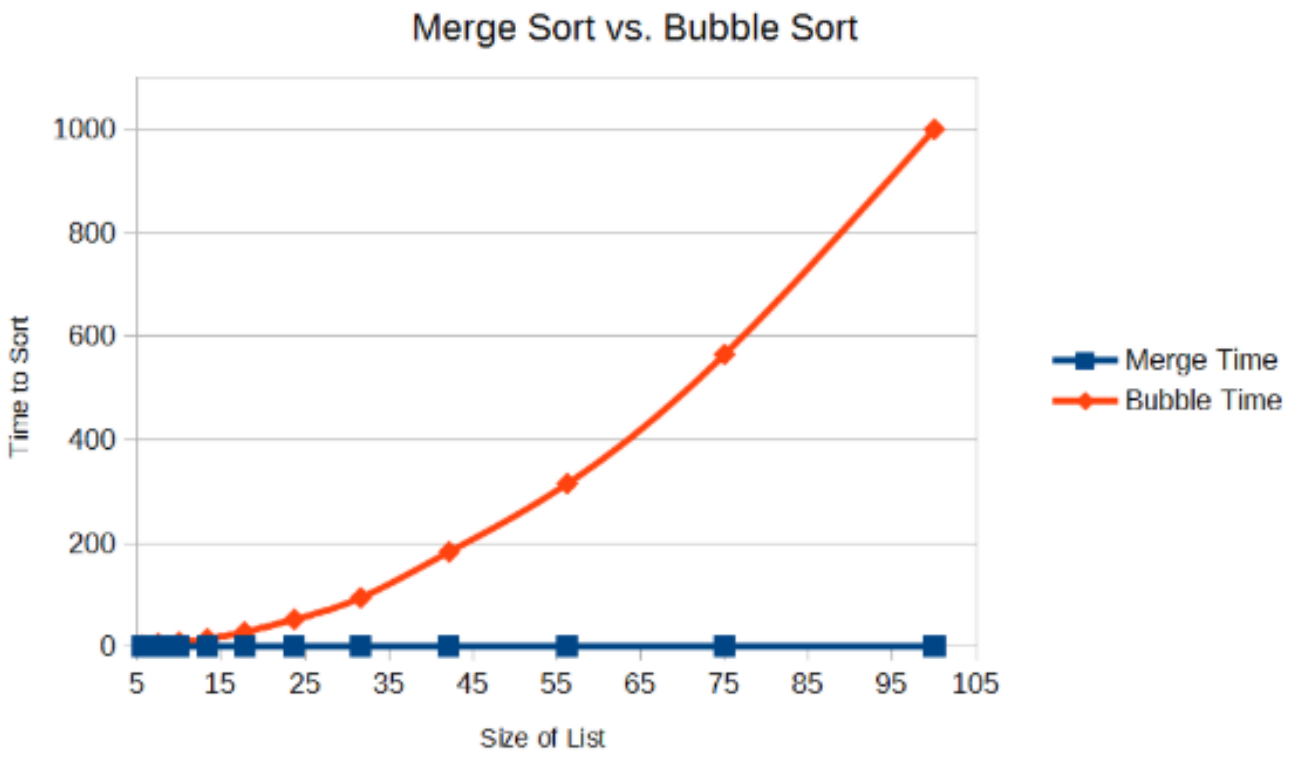
Computer Name	B
Execution rate	Ten million instructions per second
Algorithm	Merge sort ($50\, n \log n$)
Numbers to sort	one million

How do we compare two algorithms that solve the same problem?

Naïve Approach:

implement in a programming language,
ex: (C++)

run to compare their time requirements.



High specs computer doesn't help for slow algorithm and big amount of input !

Difficulties :

- *How are the algorithms coded?*
  - Independent of a particular programming style
- *What computer should we use?*
  - Independent of a particular computer specs
- *What data should the program use?*
  - Independent of specific data

Analysis of Algorithms :

analyze the efficiency of different algorithms objectively

- *Correctness*
- *Amount of Work Done (time complexity)*
- *Amount of Space Used (space complexity)*
- *Simplicity/Clarity*
- *Optimality*

Kontrak Perkuliahan

The Execution Time of Algorithms

- Each operation in an algorithm (or a program) has a cost.

  ➔ Each operation takes a certain of time.

`count = count + 1;` ➔ take a certain amount of time, but it is constant

A sequence of operations:

```
count = count + 1;        Cost: c1
sum = sum + count;        Cost: c2
```

➔ Total Cost = $c_1 + c_2$

The Execution Time of Algorithms

Example: Simple If-Statement

	Cost	Times
`if (n < 0)`	c1	1
`  absval = -n`	c2	1
`else`		
`  absval = n;`	c3	1

Total Cost = c1 + max(c2,c3)

The Execution Time of Algorithms

Example: Simple Loop

	Cost	Times
`i = 1;`	c1	1
`sum = 0;`	c2	1
`while (i <= n) {`	c3	n+1
`    i = i + 1;`	c4	n
`    sum = sum + i;`	c5	n
`}`		

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*c5

➔ The time required for this algorithm is proportional to n

The Execution Time of Algorithms

Example: Nested Loop

```
                                    Cost        Times
i=1;                                c1          1
sum = 0;                            c2          1
while (i <= n) {                    c3          n+1
     j=1;                           c4          n
     while (j <= n) {               c5          n*(n+1)
          sum = sum + i;            c6          n*n
          j = j + 1;                c7          n*n
    }
    i = i +1;                       c8          n
}
```

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5+n*n*c6+n*n*c7+n*c8

➔ The time required for this algorithm is proportional to n^2

“The running time of an algorithm typically grows with the input size.”

Analysis of Algorithms must be independent of specific implementations, computers, or data!

Employ mathematical techniques

FIRST
Prove the correctness

- *Counter example*
- *Contradiction*
- *Induction*

Second
count the number of significant operations

- *Each operation takes a certain of time*
- *Each operation in an algorithm (or a program) has a cost.*

Third
express the efficiency of algorithms using growth of functions (growth rates)

- *The cost grows with the input size.*

Tujuan Kuliah Pertemuan 1

Mahasiswa dapat memahami penggunaan konsep dasar matematika
dalam menganalisis algoritme.

Logika matematika, prinsip2 pembuktian, aljabar, sekuens, notasi sigma, peluang, aritmatika untuk mendapatkan solusi analisis algoritme.

Notasi Interval

Interval atau selang menandakan seluruh bilangan yang berada di antara dua buah nilai *a* dan *b*.

Selang buka (*a*, *b*): nilai *a* dan *b* tidak termasuk pada selang.

Selang tutup [*a*, *b*]: nilai *a* dan *b* termasuk pada selang.

[*a*, *b*) --> *a* termasuk namun *b* tidak termasuk dalam selang.

(*a*, *b*] --> *a* tidak termasuk namun *b* termasuk dalam selang.

Pertidaksamaan $3 \leq x < 8$ dapat dituliskan sebagai...

A. $(3,8)$ D. $(3,8]$ B. $[3,8)$ E. $[3,\infty)$

C. $[3,8]$

Selang $(-6,5]$ dapat ditulis sebagai pertidaksamaan...

A. $-6 < x < 5$ D. $-6 \leq x \leq 5$

B. $-6 < x \leq 5$ E. $-6 < x < \infty$

C. $-6 \leq x < 5$

Aljabar

1. $x^a x^b = x^{a+b}$
2. $x^a / x^b = x^{a-b}$
3. $(x^a)^b = x^{ab}$
4. $x^a + x^a = 2\,x^a \neq x^{2a}$
5. $\log(ab) = \log a + \log b$
6. $\log_a b = \log_k b / \log_k a$
7. $\log a^n = n \log a$

Soal 1:

Sederhanakan ekspresi berikut menggunakan sifat-sifat eksponen:

$$\frac{x^7 \cdot x^3}{x^4}$$

Jawaban:

Gunakan sifat-sifat eksponen:

$$x^7 \cdot x^3 = x^{7+3} = x^{10}$$

Kemudian, kita sederhanakan ekspresi menggunakan sifat pembagian eksponen:

$$\frac{x^{10}}{x^4} = x^{10-4} = x^6$$

Jawaban: x^6

Aljabar

1. $x^a x^b = x^{a+b}$
2. $x^a / x^b = x^{a-b}$
3. $(x^a)^b = x^{ab}$
4. $x^a + x^a = 2\,x^a \neq x^{2a}$
5. $\log(ab) = \log a + \log b$
6. $\log_a b = \log_k b / \log_k a$
7. $\log a^n = n \log a$

Soal 2:

Sederhanakan ekspresi berikut menggunakan sifat-sifat eksponen:

$$\left(x^2\right)^3 \cdot \left(x^4\right)^2$$

Jawaban:

Gunakan sifat eksponen $\left(x^a\right)^b = x^{a \cdot b}$:

$$\left(x^2\right)^3 = x^{2 \cdot 3} = x^6$$

$$\left(x^4\right)^2 = x^{4 \cdot 2} = x^8$$

Kemudian, kalikan hasilnya:

$$x^6 \cdot x^8 = x^{6+8} = x^{14}$$

Jawaban: x^{14}

Aljabar

1. $x^a x^b = x^{a+b}$
2. $x^a / x^b = x^{a-b}$
3. $(x^a)^b = x^{ab}$
4. $x^a + x^a = 2\,x^a \neq x^{2a}$
5. $\log(ab) = \log a + \log b$
6. $\log_a b = \log_k b / \log_k a$
7. $\log a^n = n \log a$

Soal 4:

Diberikan $\log_3(81) = 4$ dan $\log_3(27) = 3$. Gunakan sifat logaritma untuk menemukan $\log_9(243)$.

Jawaban:

Gunakan sifat logaritma $\log_a(b) = \frac{\log_k(b)}{\log_k(a)}$.

Karena $\log_9(243) = \frac{\log_3(243)}{\log_3(9)}$ dan $9 = 3^2$, kita tahu bahwa:

$$\log_3(9) = 2$$

Sekarang, $243 = 3^5$, sehingga:

$$\log_3(243) = 5$$

Jadi:

$$\log_9(243) = \frac{5}{2}$$

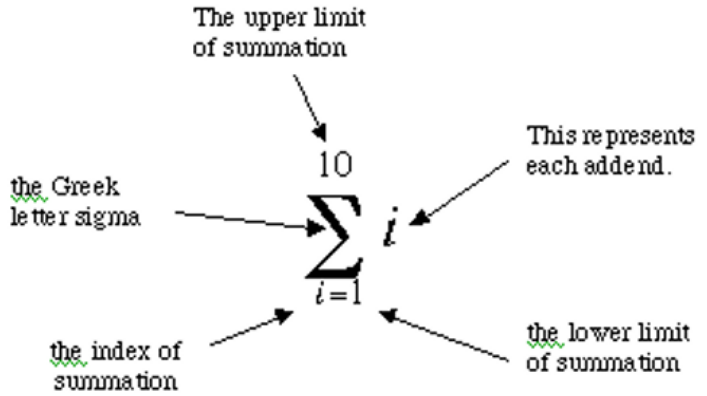
Barisan dan Deret

Barisan: kumpulan nilai setiap sukunya didapatkan dari suku sebelumnya.

- **Barisan aritmatika**: suatu suku diperoleh dengan menambahkan suatu bilangan *d* pada suku sebelumnya.
  - Suku ke-n: $a_n = a+(n-1)d$
  - Contoh: 2, 4, 6, 8, ... ($d = 2$)
- **Barisan geometri**: suatu suku diperoleh dengan mengalikan suatu bilangan *r* pada suku sebelumnya.
  - Suku ke-n: $a_n = ar^{n-1}$
  - Contoh: 2, 4, 8, 16, …. ($r = 2$)

Deret: jumlah setiap suku dalam suatu barisan.

- **Barisan aritmatika**: $S_n = n(2a + (n-1)d) / 2$
- **Barisan geometri**: $S_n = a(r^n - 1) / (r - 1)$



$\sum i = n(n+1)/2 \approx \frac{1}{2} n^2$

$\sum i^2 = n(n+1)(2n+1)/6 \approx 1/3\ n^3$

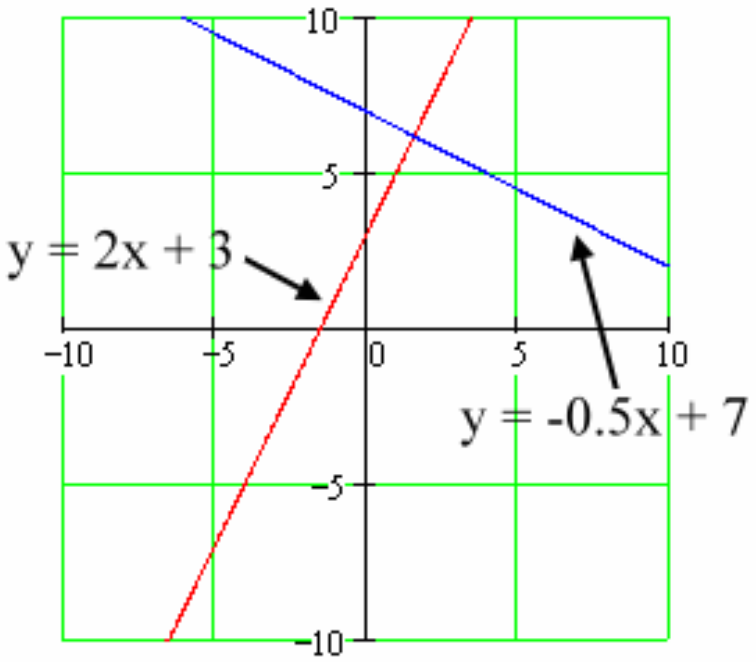
Faktorial, Permutasi, Kombinasi

- **Notasi Faktorial**:
  - $n!$ adalah hasil perkalian n bilangan bulat pertama.
  - $n! = 1 * 2 * 3 * ... * (n-1) * n$
  - Secara definisi, 0! bernilai 1.
- **Kombinotarika**
  - **Prinsip Penjumlahan**: jika cara untuk melakukan A adalah *m* dan melakukan B adalah *n*, maka jumlah cara untuk melakukan A atau B adalah *m*+*n*.
  - **Prinsip Perkalian**: jika cara untuk melakukan A adalah *m* dan melakukan B adalah *n*, maka jumlah cara untuk melakukan A dan B (melakukan A lalu melakukan B) adalah *m***n*.
  - Misal *n* adalah jumlah objek yang dapat dipilih dan *r* adalah jumlah yang harus dipilih.
  - **Permutasi**: menggabungkan beberapa objek dari suatu grup dengan memperhatikan urutan.
    - $P(n, r) = n!/(n-r)!$
  - **Kombinasi**: menggabungkan beberapa objek dari suatu grup tanpa memperhatikan urutan.
    - $C(n, r) = n!/((n-r)!r!)$

Faktorial, Permutasi, Kombinasi

- **Notasi Faktorial**:
  - $n!$ adalah hasil perkalian n bilangan bulat pertama.
  - $n! = 1 * 2 * 3 * ... * (n-1) * n$
  - Secara definisi, 0! bernilai 1.
- **Kombinotarika**
  - **Prinsip Penjumlahan**: jika cara untuk melakukan A adalah *m* dan melakukan B adalah *n*, maka jumlah cara untuk melakukan A atau B adalah *m*+*n*.
  - **Prinsip Perkalian**: jika cara untuk melakukan A adalah *m* dan melakukan B adalah *n*, maka jumlah cara untuk melakukan A dan B (melakukan A lalu melakukan B) adalah *m***n*.
  - Misal *n* adalah jumlah objek yang dapat dipilih dan *r* adalah jumlah yang harus dipilih.
  - **Permutasi**: menggabungkan beberapa objek dari suatu grup dengan memperhatikan urutan.
    - $P(n, r) = n!/(n-r)!$
  - **Kombinasi**: menggabungkan beberapa objek dari suatu grup tanpa memperhatikan urutan.
    - $C(n, r) = n!/((n-r)!r!)$

Perpotongan Dua Fungsi



- Diberikan dua buah fungsi linear, carilah seluruh titik tempat kedua fungsi tersebut berpotongan.
- Ada tiga cara yang bisa digunakan: subtitusi, eliminasi, dan determinan.

Perpotongan Dua Fungsi: Substitusi

1. Metode Substitusi

Untuk metode substitusi, kita set kedua persamaan fungsi sama dengan y:

$$2x + 3 = -0.5x + 7$$

Sekarang, kita selesaikan untuk x:

$$2x + 0.5x = 7 - 3$$

$$2.5x = 4$$

$$x = \frac{4}{2.5} = 1.6$$

Setelah mendapatkan nilai x, kita substitusikan kembali ke salah satu persamaan untuk mendapatkan y:

$$y = 2(1.6) + 3 = 3.2 + 3 = 6.2$$

Jadi, titik potongnya adalah $(1.6, 6.2)$.

Logika Matematika

- Proposisi
- Perangkai dasar
- Tabel kebenaran
- Dalil-dalil kesetaraan
- Kesetaraan proposisi kompleks
- Argumen
- Kaidah inferensia : modus ponen, modus tolens, kaidah silogisma

Manakah yg merupakan Proposisi?

Proposisi : pernyataan atau kalimat deklaratif yang bernilai benar (true) atau salah (false), tetapi tidak keduanya.

- “Jerapah lebih besar daripada semut.” √
- *300>200* √
- ~~*x>500*~~
- “Grass is green“ √
- "2 + 5 = 5“ √
- ~~"Close the door“~~
- ~~"Is it hot outside ?“~~
- ~~"x is greater than 2"~~

Perangkai Dasar

Perangkai dasar (beserta simbol):

1. "**dan**",diberi simbol khusus "∧"
2. "**atau**" ,diberi simbol khusus "v"
3. "**tidak/bukan**" ,diberi simbol "~"
4. "**jika...,maka...**" ,diberi simbol "→"
5. "**jika dan hanya jika**" ,diberi simbol "↔

Tabel Kebenaran

P	q	~p	~q	p ∧ q	p ∨ q	p → q	p ↔ q
B	B	S	S	B	B	B	B
B	S	S	B	S	B	S	S
S	B	B	S	S	B	B	S
S	S	B	B	S	S	B	B

Dalil-dalil Kesetaraan

Dalil Identitas

a. $p \vee 0 = p$
b. $p \vee 1 = 1$
c. $p \wedge 0 = 0$
d. $p \wedge 1 = p$

Dalil Idempoten

a. $p \vee p = p$
b. $p \wedge p = p$

Dalil *de Morgan*

a. $-(p \vee q) = -p \wedge -q$
b. $-(p \wedge q) = -p \vee -q$

Dalil Komplemen

a. $p \vee -p = 1$
b. $p \wedge -p = 0$

Dalil Komutatif

a. $p \vee q = q \vee p$
b. $p \wedge q = q \wedge p$

Dalil Distributif

a. $p \vee (q \wedge r) = (p \vee q) \wedge (p \vee r)$
b. $p \wedge (q \vee r) = (p \wedge q) \vee (p \wedge r)$

Dalil Asosiatif

a. $(p \vee q) \vee r = p \vee (q \vee r)$
b. $(p \wedge q) \wedge r = p \wedge (q \wedge r)$

Dalil Ingkaran Ganda

$-(-p) = p$

Dalil Penghapusan

a. $(p \vee q) \wedge p = p$
b. $(p \wedge q) \vee q = q$

Kesetaraan lain yang digunakan :

a. $p \rightarrow q = -q \rightarrow -p$
b. $p \rightarrow q = -p \vee q$
c. $-(p \rightarrow q) = p \wedge -q$
d. $p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p) = (-p \vee q) \vee (-q \vee p)$

Argumen

Argumen terdiri dari pernyataan-pernyataan yang terdiri atas dua kelompok, premis (pernyataan) dan konklusi (kesimpulan)

Ani ada di Bandung atau Tasikmalaya. Ani tidak ada di Bandung. Jadi, Ani ada di Tasikmalaya.

Misal:

p : Ani ada di Bandung
q : Ani ada di Tasikmalaya
p V q
~p
∴q

Rules of Inference/Kaidah Inferensia

- Modus ponen

Premis 1 :	$a \Rightarrow b$
Premis 2 :	a
Kesimpulan :	$\therefore b$

Jika saya makan di kelas maka saya minum di kelas. Saya makan di kelas.

saya minum di kelas.

- Modus tolens

Premis 1 :	$a \Rightarrow b$
Premis 2 :	$\sim b$
Kesimpulan :	$\therefore \sim a$

Jika saya makan di kelas maka saya minum di kelas. Saya tidak minum di kelas.

saya tidak makan di kelas.

- Silogisme

Premis 1 :	$a \Rightarrow b$
Premis 2 :	$b \Rightarrow c$
Kesimpulan :	$\therefore a \Rightarrow c$

Jika saya makan di kelas, maka kelas akan kotor. Jika kelas kotor, maka saya dimarahi guru.

Jika saya makan di kelas, maka saya dimarahi guru.

Metode Pembuktian Counter Example

- Counter Example adalah metode pembuktian di mana kita menunjukkan bahwa pernyataan dengan bentuk "jika p maka q" adalah **salah / tidak benar**
- Caranya dengan memberikan minimal satu contoh spesifik di mana p benar tetapi q salah.

If p is prime, then $M_p = 2^p - 1$ is prime

Prime → 2,3,5,7,11,13……

$$M_2 = 2^2 - 1 = 3$$
$$M_3 = 2^3 - 1 = 7$$
$$M_5 = 2^5 - 1 = 31$$
$$M_7 = 2^7 - 1 = 127$$
$$M_{11} = 2^{11} - 1 = 2047 \longrightarrow$$ It's not prime

If p is prime, then $M_p = 2^p - 1$ is not always prime

Metode Pembuktian Direct Proof

- A direct proof is one of the most familiar forms of proof.
- We use it to prove statements of the form ”if p then q” or ”p implies q” which we can write as p ⇒ q.
- The method of the proof is to takes an original statement p, which we assume to be true, and use it to show directly that another statement q is true.
- Direct proof tidak bisa digunakan untuk membuktikan pernyataan yang salah

Proposition : If P then Q
Proof : Suppose P
:
:
Therefore Q

So a direct proof has the following steps:

1. **Assume the statement p is true.**
2. **Use what we know about p (ex: definition) and other facts as necessary to deduce that another statement q is true, that is show p ⇒ q is true.**

Direct Proof Example

Proposition : If P then Q
Proof : Suppose P
:
:
Therefore Q

Proposition : If n is even then n^2 is even
P : n is even
Q : n^2 is even

Proof : Suppose n is even

Definition : an integer n is even if $n = 2a$ for some integer a

So, $n = 2a$ for some integer a

Consider n^2

$$n^2 = n.n = 2a.2a = 2(2a^2) = 2w$$

where for some integer,

$$w = 2a^2$$

Therefore n^2 is even

Metode Pembuktian Indirect Proof

- Ada dua jenis pembuktian tidak langsung: pembuktian dengan kontrapositif dan pembuktian dengan kontradiksi.
- Dalam pembuktian dengan kontrapositif, kita sebenarnya menggunakan pembuktian langsung untuk membuktikan kontrapositif dari implikasi asli.
- Dalam pembuktian dengan kontradiksi, kita memulai dengan anggapan bahwa implikasi tersebut salah, dan menggunakan asumsi ini untuk mendapatkan sebuah kontradiksi.
- Hal ini akan membuktikan bahwa implikasinya adalah benar.

Metode Pembuktian
Indirect Proof : Contrapositive

- The expression $\sim Q \Rightarrow \sim P$ is called the contrapositive form of $P \Rightarrow Q$
- **use direct proof** to show $\sim Q \Rightarrow \sim P$ is true, we would assume $\sim Q$ is true and use this to deduce that $\sim P$ is true.

So proofing with contrapositive has the following steps:

Proposition : If P then Q
Proof : Suppose $\sim Q$
:
:
Therefore $\sim P$

Contrapositive Example

Proposition : If P then Q
Proof : Suppose ~ Q
:
:
Therefore ~ P

Proposition : If n^2 is even then n is even
P : n^2 is even
Q : n is even

Proof : Suppose n is not even (then n is odd)

Definition : an integer n is odd if $n = 2a + 1$ for some integer a
So, $n = 2a + 1$ for some integer a

Consider n^2

$$n^2 = (2a + 1)^2 = 4a^2 + 4a + 1 = 2(2a^2 + 2a) + 1 = 2w + 1$$

where for some integer,

$$w = (2a^2 + 2a)$$

Therefore n^2 is not even
So the statement "If n^2 is even then n is even" is true

Metode Pembuktian
Indirect Proof : Contradiction

- Assume the opposite of statement you want to prove is true and then show that this assumption leads to a logical contradiction.
- This contradiction proves that the original assumption must be false, and therefore the desired statement is true.

A proof by contradiction is based on the fact that $P \Rightarrow Q$ is equivalent to $(\sim P \vee Q)$, an equivalency demonstrated earlier. Recall that the negation of $(\sim P \vee Q)$ is $(P \wedge \sim Q)$. If we can show that this negation is false, then the statement, $(\sim P \vee Q)$, is true and hence so is the equivalent statement, $P \Rightarrow Q$.

Contradiction Example

Proposition : If P then Q
Proof : Suppose ~(P→Q)
:
:
Therefore C ∧ ~C



Proposition : If P then Q
Proof : Suppose P ∧ ~Q

:
:
Therefore C ∧ ~C

Proposition : If n^2 is even then n is even
P : n^2 is even
Q : n is even

Proof : Suppose n^2 **is even** and n is odd

Definition : an integer n is odd if $n = 2a + 1$ for some integer a
So, $n = 2a + 1$ for some integer a

Consider n^2

$$n^2 = (2a + 1)^2 = 4a^2 + 4a + 1 = 2(2a^2 + 2a) + 1 = 2w + 1$$

where for some integer,

$$w = (2a^2 + 2a)$$

Therefore n^2 **is not even** → **contradiction**

So the statement "If n^2 is even then n is even" is true

CORRECTNESS

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Induction!



The base case. P(0) is shown to be true



The induction hypothesis. P(n) is assumed to be true for some arbitrary natural number n.



The induction step. Using the induction hypothesis, P(n+1) is proved.

Why Induction?

```
x = c; y = 0;
while (x > 0) {
  x--;
  y++;
}
```

Property: y equals c after the loop terminates

Strategy:
Compute state after iteration #1, iteration #2, …
Prove that state after last iteration has y = c

Better Strategy:
Use induction (over number of iterations)
Base case: Prove induction hypothesis holds on loop entry
Induction case: Assuming induction hypothesis holds after k iterations, prove it holds after k+1 iterations

Proof by Loop Invariant Is...

Invariant: something that is always true

After finding a candidate loop invariant, we prove:

1. *Initialization*: How does the invariant get initialized?
2. *Loop Maintenance*: How does the invariant change at each pass through the loop?
3. *Termination*: Does the loop stop? When?

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

TUGAS 1

Lakukan pembuktian secara induksi menggunakan loop invariant :

```
x = c; y = 0;
while (x > 0) {
   x--;
   y++;
}
```

SQUARE Function: SQ(n)

```
S <- 0
i <- 0
while i < n
   S <- S + n
   i <- i + 1
return S
```

FACTORIAL Function: FAC(n)

```
i <- 1
F <- 1
while i < = n
   F <- F * i
   i <- i + 1
return F
```

Dikerjakan secara individu menggunakan tulisan tangan dan difoto/discan, dikumpulkan saat praktikum dalam format PDF.

Example Problem 1.

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Step 1: Construct an Inductive Hypothesis

Property: y equals c after the loop terminates

```
x = c; y = 0;
while (x > 0) {
  x--;
  y++;
}
```

We can generalize from examples…

- On loop entry: x = c, y = 0
- After iteration 1: x = c - 1, y = 1
- After iteration 2: x = c - 2, y = 2
- …

inductive hypothesis
$x + y = c$

Inductive Hypothesis is the loop invariant!!!

Loop Invariant is some predicate (condition) that holds for every iteration of the loop.

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Step 2: Prove that Loop Invariant is Inductive

```
x = c; y = 0;
while (x > 0) {
  x--;
  y++;
}
```

We can generalize from examples…

- On loop entry: x = c, y = 0
- After iteration 1: x = c - 1, y = 1
- After iteration 2: x = c - 2, y = 2
- …

inductive hypothesis

x + y = c

x=c-k, y=k

1. Base case: loop invariant x + y = c holds on loop entry
   True
2. Inductive case:
   Assume loop invariant holds after k iterations:
   y = k, x = c - k
   After the (k+1)st iteration, y = k + 1, x = c - k - 1
   Therefore, x + y = k + 1 + c - k - 1 = c
   True

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Step 3: Proving correctness property using loop invariant

```
x = c; y = 0;
while (x > 0) {
  x--;
  y++;
}
```

We can generalize from examples…

- On loop entry: x = c, y = 0
- After iteration 1: x = c - 1, y = 1
- After iteration 2: x = c - 2, y = 2
- …

inductive hypothesis

x + y = c

Property: y equals c after the loop terminates

- Use loop invariant to prove correctness property that y = c after loop terminates

Now, when the algorithm stops : x = 0

We also know our loop invariant holds: x + y = c

Therefore, y = c.

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Example Problem 2.

https://inst.eecs.berkeley.edu/~cs170/fa14/tutorials/tutorial1.pdf

Step 1: Construct an Inductive Hypothesis

SQUARE Function: SQ(n)

```
S <- 0
i <- 0
while i < n
   S <- S + n
   i <- i + 1
return S
```

Correctness Property :
S=n*n
after loop terminates

Generalize from examples, n = 3

After iteration k=1
First S <- 0 and i <- 0 give S = 0 and i = 0 initially.
Since i < 3, the while loop is entered.
S <- 0 + 3
i <- 0 + 1
producing **S = 3 and i = 1.**

After iteration k=2
Since i < 3, the while loop is entered the second time.
S <- 3 + 3
i <- 1 + 1
producing **S = 6 and i = 2.**

After iteration k=3
Since i < 3, the while loop is entered the third time.
S <- 6 + 3
i <- 2 + 1
producing **S = 9 and i = 3.**
Since i = 3, the while loop is not entered any longer, S = 9 is returned and the algorithm is terminted.

Inductive hypothesis

S = k*n and i = k hold

Loop Invariant

Step 2: Prove that Loop Invariant is Inductive

SQUARE Function: SQ(n)

```
S <- 0
i <- 0
while i < n
   S <- S + n
   i <- i + 1
return S
```

Correctness Property :
S=n*n
after loop terminates

Proof by induction.

1. Base case : k = 0. When k = 0, that is when the loop is not entered, S = 0 and i = 0.
   Hence S = k*n and i = k hold.
2. Inductive Hypothesis: For an arbitrary value m of k, S = m * n and i = m hold after going through the loop m times.
3. Inductive Step: When the loop is entered (m + 1)-st time, S = m*n and i = m at the beginning of the loop.
   Inside the loop,
   S <- m*n + n
   i <- i + 1
   producing S = (m + 1)*n and i = m + 1.
   Thus S = k*n and i = k hold for any natural number k.

Step 3: Proving correctness property using loop invariant

SQUARE Function: SQ(n)

```
S <- 0
i <- 0
while i < n
   S <- S + n
   i <- i + 1
return S
```

Correctness Property :
S=n*n
after loop terminates

Property : S=n*n after loop terminates

Now, when the algorithm stops, i = n.
Hence the loop will have been entered n times.
Loop invariant : **S = k*n and i = k hold** → i = n, thus S = n*n = n^2
Hence the algorithm is correct.

Example Problem 3.

Step 1: Construct an Inductive Hypothesis

FACTORIAL Function: FAC(n)

```
i <- 1
F <- 1
while i < = n
   F <- F * i
   i <- i + 1
return F
```

Correctness Property : F=n! after loop terminates

Generalize from examples, n = 3

After iteration k=1

First i <- 1 and F <- 1 give i = 1 and F = 1 initially.
Since i < 3, the while loop is entered.
F <- 1 * 1
i <- 1 + 1
producing **F = 1 and i = 2**.

After iteration k=2

Since i < 3, the while loop is entered the second time.
F <- 1 * 2
i <- 2 + 1
producing **F = 2 and i = 3**.

After iteration k=3

Since i = 3, the while loop is entered the third time.
F <- 2 * 3
i <- 3 + 1
producing **F = 6 and i = 4.**
Since i = 4, the while loop is not entered any longer, F = 6 is returned and the algorithm is terminted.

Inductive hypothesis

F = k ! and i = k + 1 hold

Loop Invariant

Step 2: Prove that Loop Invariant is Inductive

FACTORIAL Function: FAC(n)

```
i <- 1
F <- 1
while i < = n
   F <- F * i
   i <- i + 1
return F
```

Correctness Property : F=n! after loop terminates

Loop Invariant
F = k ! and i = k + 1 hold

Proof by induction.

1. Base Case : k = 1. When k = 1, that is when the loop is entered the first time.
   F = 1 * 1 = 1 and i = 1 + 1 = 2. Since 1! = 1, F = k! and i = k + 1 hold.
2. Inductive Hypothesis : For an arbitrary value , k=m, F = m! and i = m + 1 hold after going through the loop m times.
3. Inductive Step: When the loop is entered (m + 1)-st time, F = m! and i = (m+1) at the beginning of the loop.

Inside the loop,

F <- m!* (m + 1)
i <- (m + 1) + 1

producing F = (m + 1)! and i = (m + 1) + 1.
Thus F = k! and i = k + 1 hold for any positive integer k.

Step 3: Proving correctness property using loop invariant

FACTORIAL Function: FAC(n)

```
i <- 1
F <- 1
while i < = n
   F <- F * i
   i <- i + 1
return F
```

Correctness Property :
F=n! after loop terminates

Loop Invariant
F = k ! and i = k + 1 hold

Now, when the algorithm stops, i = n + 1.
Hence the loop will have been entered n times→k=n.
Thus F = n! is returned.
Hence the algorithm is correct.

Thank You

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam tincidunt ante nec sem tincidunt ante nec.

Modern Presentation Modern Presentation